

Practical Kubernetes Administration and Troubleshooting

Day 2: Core Concepts and Workload Management

Instructor: Chad M. Crowell

Day 1 Knowledge Check

5 quick questions before we dive into Day 2

Quiz: Day 1 Recap

1. What is the **only** control plane component that communicates directly with etcd?
2. Which Kubernetes interface – CRI, CNI, or CSI – is responsible for assigning a Pod its IP address?
3. What command initializes the control plane node with kubeadm?
4. True or False: Kubernetes ships with a default CNI plugin out of the box.
5. When a component **connects** to another component (e.g. kubelet → API server), what type of certificate does it present to authenticate itself?

Quiz: Answers

1. **The API Server** – all other components (scheduler, kubelet, controller manager) talk to etcd only through it
2. **CNI** – the Container Network Interface plugin assigns the Pod IP via its IPAM plugin
3. `kubeadm init --pod-network-cidr=<cidr> --kubernetes-version=stable`
4. **False** – Kubernetes does not ship with a CNI plugin; you must install one yourself (e.g. Calico)
5. **A client certificate** – verified by the receiving component against the cluster CA (mTLS)

Day 1 Recap

What We Covered Yesterday

- Containers vs Virtual Machines – the evolution of app deployment
- Kubernetes Architecture – control plane, worker nodes, and how they interact
- Component Communication & TLS – mTLS, certificates, cluster PKI
- Kubernetes Interfaces – CRI, CNI, and CSI
- Installed a cluster from scratch with kubeadm
- Validated nodes, components, and networking

Where We Left Off

- A working cluster with 1 control plane + 1 worker
- kubectl configured and communicating with the API server
- Calico CNI installed, all nodes Ready

Day 2 Agenda

Time	Topic
Morning	Namespaces and Resource Organization
	Pods, ReplicaSets, and Deployments
	Scaling Applications
	Declarative vs Imperative Management
Afternoon	ConfigMaps and Secrets
	Health Checks: Liveness and Readiness Probes
	Common Troubleshooting Commands
	Hands-On Labs

Labs Today

- Deploy a multi-replica application with Deployments
- Perform rolling updates and rollbacks
- Create and consume ConfigMaps and Secrets
- Configure liveness and readiness probes
- Troubleshoot broken pods

Namespaces

Organizing resources in a Kubernetes cluster

What Are Namespaces?

Namespaces provide a way to **divide cluster resources** between multiple users, teams, or environments.

Why Use Namespaces?

- **Isolation** – separate environments (dev, staging, prod)
- **Access control** – RBAC policies scoped to a namespace
- **Resource quotas** – limit CPU/memory per namespace
- **Organization** – avoid naming collisions across teams

Default Namespaces

Namespace	Purpose
default	Where resources go if no namespace is specified
kube-system	Control plane components (API server, scheduler, etc.)
kube-public	Publicly readable data (e.g. cluster-info ConfigMap)
kube-node-lease	Node heartbeat leases for health detection

Working with Namespaces

Create a Namespace

```
kubectl create namespace dev
```

Or declaratively:

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

Target a Namespace

```
# list pods in the dev namespace
kubectl get pods -n dev

# list pods across ALL namespaces
kubectl get pods -A
```

Setting a Default Namespace

Tired of typing `-n dev` on every command? Set a default namespace for your context:

```
# set the default namespace for the current context
kubectl config set-context --current --namespace=dev
```

Verify

```
kubectl config view --minify | grep namespace
```

Now all `kubectl` commands will target `dev` unless you specify `-n` otherwise.

Tip: Not all resources are namespaced. Nodes, PersistentVolumes, and ClusterRoles are **cluster-scoped**. Use `kubectl api-resources --namespaced=false` to see the full list.

Pods, ReplicaSets, and Deployments

The core workload resources

Pods – Revisited

A **Pod** is the smallest deployable unit in Kubernetes – one or more containers that share networking and storage.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
```

Key Facts

- Pods are **ephemeral** – they are not rescheduled if a node goes down
- Pods get a **unique IP** address within the cluster
- Containers in a pod share `localhost` and can communicate via IPC
- **Labels** are key-value pairs used to organize and select pods

You rarely create bare Pods in production – you use a controller (Deployment, ReplicaSet) to manage them.

Labels and Selectors

Labels are the **backbone of how Kubernetes connects resources** together.

Adding Labels

```
metadata:
  labels:
    app: frontend
    env: production
    version: v2
```

Selecting by Label

```
# find all pods with app=frontend
kubectl get pods -l app=frontend

# find pods matching multiple labels
kubectl get pods -l "app=frontend,env=production"

# find pods where env is NOT production
kubectl get pods -l "env≠production"
```

Labels are how Services find pods, how Deployments manage ReplicaSets, and how network policies target workloads. Master them early.

ReplicaSets

A **ReplicaSet** ensures that a specified number of pod replicas are running at any given time.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

How It Works

- The **selector** tells the ReplicaSet which pods to manage
- The **template** defines what new pods look like
- If a pod is deleted, the ReplicaSet creates a replacement

Key Fields

Field	Purpose
replicas	Desired number of pods
selector	Label query to find managed pods
template	Pod spec for new replicas

In practice, you almost never create ReplicaSets directly – Deployments manage them for you.

Deployments

A **Deployment** is the recommended way to manage stateless applications. It manages ReplicaSets, which manage Pods.

Deployment → ReplicaSet → Pod, Pod, Pod

Why Deployments Over ReplicaSets?

- **Rolling updates** – gradually replace old pods with new ones
- **Rollbacks** – revert to a previous version instantly
- **Revision history** – track changes over time
- **Declarative updates** – change the spec, Kubernetes handles the rest

Creating a Deployment: Imperative

```
kubectl create deployment nginx --image=nginx:1.27 --replicas=3
```

Creating a Deployment: Declarative

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

Inspecting a Deployment

```
# see deployment status
kubectl get deployments

# detailed info including strategy, conditions, and events
kubectl describe deployment nginx

# see the ReplicaSets managed by this Deployment
kubectl get rs

# see the pods created by the ReplicaSet
kubectl get pods --show-labels
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	2m

The Relationship

<code>kubectl get deploy nginx</code>	→ Deployment (desired state)
<code>kubectl get rs</code>	→ ReplicaSet (current generation)
<code>kubectl get pods</code>	→ Pods (actual running containers)

Rolling Updates

When you update a Deployment's pod template (e.g. change the image), Kubernetes performs a **rolling update**.

Update the Image

```
kubectl set image deployment/nginx nginx=nginx:1.28
```

Watch the Rollout

```
kubectl rollout status deployment/nginx
```

What Happens During a Rolling Update

1. A **new ReplicaSet** is created with the updated pod template
2. New pods are gradually scaled **up**
3. Old pods are gradually scaled **down**
4. At no point are all pods unavailable (zero-downtime)

```
# see both ReplicaSets during the transition  
kubectl get rs -w
```

Rolling Update Strategy

The Deployment spec controls **how fast** pods are replaced:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1    # at most 1 pod can be down during update
      maxSurge: 1         # at most 1 extra pod above desired count
```

Parameter	Meaning
maxUnavailable	Max pods that can be unavailable during the update
maxSurge	Max pods that can exist above the desired replica count

The Other Strategy: Recreate

```
spec:
  strategy:
    type: Recreate    # kills ALL old pods first, then creates new ones
```

Use `Recreate` when your app **cannot** run two versions simultaneously (e.g. database migrations, license constraints).

Rollbacks

Every Deployment update creates a new **revision**. You can roll back to any previous revision.

View Rollout History

```
kubectl rollout history deployment/nginx
```

REVISION	CHANGE-CAUSE
1	<none>
2	<none>
3	<none>

Roll Back to the Previous Version

```
kubectl rollout undo deployment/nginx
```

Roll Back to a Specific Revision

```
kubectl rollout undo deployment/nginx --to-revision=1
```

Tip: Add `--record` to your commands (deprecated but still works) or use annotations to track change causes: `kubectl annotate deployment/nginx kubernetes.io/change-cause="Updated to nginx 1.28"`

Scaling Applications

Manual and automatic scaling

Manual Scaling

Scale a Deployment

```
# scale to 5 replicas
kubectl scale deployment/nginx --replicas=5

# verify
kubectl get deployment nginx
kubectl get pods
```

Scale to Zero

```
# effectively "pause" a workload without deleting it
kubectl scale deployment/nginx --replicas=0
```

Scaling to zero is useful for cost savings in dev/staging or temporarily disabling a service.

Horizontal Pod Autoscaler (HPA)

The HPA automatically scales the number of pods based on observed **CPU utilization** or **custom metrics**.

Prerequisites

The **Metrics Server** must be installed in the cluster:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

# tell Metrics Server to skip kubelet cert validation
kubectl patch deployment metrics-server -n kube-system --type='json' \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
```

Create an HPA

```
kubectl autoscale deployment/nginx --min=2 --max=10 --cpu-percent=50
```

This creates an HPA that:

- Keeps at least **2** replicas
- Scales up to **10** replicas
- Targets **50%** average CPU utilization across all pods

HPA in YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Important: For HPA to work, your pods must have **resource requests** defined. Without them, the HPA has no baseline to calculate utilization against.

Test HPA

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
```

Resource Requests and Limits

Every container can declare how much CPU and memory it needs.

```
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    resources:
      requests:          # guaranteed minimum
        cpu: "100m"      # 100 millicores = 0.1 CPU
        memory: "128Mi"
      limits:            # maximum allowed
        cpu: "500m"
        memory: "256Mi"
```

Field	Meaning
requests	The scheduler uses this to place the pod on a node with enough capacity
limits	The kubelet enforces this – container is throttled (CPU) or killed (memory) if exceeded

Declarative vs Imperative Management

Two approaches to managing Kubernetes resources

Imperative Commands

You tell Kubernetes **what to do** step by step:

```
# create
kubectl create deployment nginx --image=nginx:1.27 --replicas=3

# update
kubectl set image deployment/nginx nginx=nginx:1.28

# scale
kubectl scale deployment/nginx --replicas=5

# delete
kubectl delete deployment nginx
```

When to Use Imperative

- Quick one-off tasks
- Exploration and debugging
- CKA exam (speed matters)

Imperative commands are fast but **not reproducible**. There's no record of what you did.

Declarative Management

You describe the **desired state** in YAML and let Kubernetes figure out how to get there:

```
# apply the desired state – creates or updates
kubectl apply -f deployment.yaml

# delete by file
kubectl delete -f deployment.yaml
```

When to Use Declarative

- Production environments
- GitOps workflows
- Anything you want to **version control**

The Key Difference

Approach	Command	Idempotent?	Trackable?
Imperative	kubectl create	No – fails if exists	No
Declarative	kubectl apply	Yes – creates or updates	Yes – YAML in git

Rule of thumb: Use imperative for learning and debugging, declarative for everything else.

Generating YAML from Imperative Commands

You don't have to write YAML from scratch. Use `--dry-run=client -o yaml` to generate it:

```
# generate a Deployment YAML without creating it
kubectl create deployment nginx --image=nginx:1.27 --replicas=3 \
  --dry-run=client -o yaml > deployment.yaml

# generate a Service YAML
kubectl expose deployment nginx --port=80 --type=ClusterIP \
  --dry-run=client -o yaml > service.yaml

# generate a ConfigMap YAML
kubectl create configmap app-config --from-literal=ENV=production \
  --dry-run=client -o yaml > configmap.yaml
```

Edit the generated YAML, then apply:

```
kubectl apply -f deployment.yaml
```

This is the fastest way to get correct YAML – let kubectl do the boilerplate, you customize the details.

ConfigMaps and Secrets

Externalizing configuration from your application

Why Externalize Configuration?

Hardcoding configuration inside container images is a problem:

- Rebuilding the image for every environment (dev, staging, prod)
- Secrets baked into images end up in registries
- No way to change config without redeploying

The Kubernetes Solution

Resource	Purpose
ConfigMap	Non-sensitive configuration data (env vars, config files)
Secret	Sensitive data (passwords, API keys, TLS certs)

Both are stored in etcd and can be consumed by pods as **environment variables** or **mounted files**.

Creating ConfigMaps

From Literals

```
kubectl create configmap app-config \  
  --from-literal=APP_ENV=production \  
  --from-literal=LOG_LEVEL=info
```

From a File

```
# create a config file  
echo "server.port=8080" > app.properties  
  
kubectl create configmap app-config --from-file=app.properties
```

Declarative YAML

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: app-config  
data:  
  APP_ENV: production  
  LOG_LEVEL: info  
  app.properties: |  
    server.port=8080  
    server.host=0.0.0.0
```

Consuming ConfigMaps

As Environment Variables

```
spec:
  containers:
  - name: app
    image: myapp:latest
    envFrom:
    - configMapRef:
        name: app-config      # injects ALL keys as env vars
```

Or select specific keys:

```
spec:
  containers:
  - name: app
    image: myapp:latest
    env:
    - name: APP_ENVIRONMENT
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: APP_ENV
```

Consuming ConfigMaps (cont.)

As a Mounted Volume

```
spec:
  containers:
    - image: myapp:latest
      name: app
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-config      # each key becomes a file in /etc/config/
```

Creating Secrets

Secrets work the same way as ConfigMaps, but values are **base64-encoded** in etcd.

From Literals

```
kubectl create secret generic db-credentials \  
  --from-literal=DB_USER=admin \  
  --from-literal=DB_PASS=s3cretP@ss
```

Declarative YAML

```
apiVersion: v1  
kind: Secret  
metadata:  
  name: db-credentials  
type: Opaque  
data:  
  DB_USER: YWRtaW4=          # echo -n "admin" | base64  
  DB_PASS: czNjcmV0UEBzcw==  # echo -n "s3cretP@ss" | base64
```

Creating Secrets (cont.)

Or use `stringData` to avoid manual base64 encoding:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  DB_USER: admin
  DB_PASS: s3cretP@ss
```


Consuming Secrets

As Environment Variables

```
spec:
  containers:
  - name: app
    image: myapp:latest
    envFrom:
    - secretRef:
        name: db-credentials
```

As a Mounted Volume

```
volumeMounts:
- name: secret-volume
  mountPath: /etc/secrets
  readOnly: true
volumes:
- name: secret-volume
  secret:
    secretName: db-credentials # each key becomes a file
```

Warning: Kubernetes Secrets are **base64-encoded, not encrypted** by default. Anyone with API access can decode them. For production, enable **encryption at rest** or use an external secret manager (Vault, AWS Secrets Manager, etc.).

Secret Types

Kubernetes has several built-in Secret types:

Type	Purpose
Opaque	Generic key-value pairs (default)
kubernetes.io/tls	TLS certificate and key
kubernetes.io/dockerconfigjson	Docker registry credentials
kubernetes.io/basic-auth	Username and password
kubernetes.io/service-account-token	Auto-generated SA tokens

TLS Secret Example

```
kubectl create secret tls my-tls-secret \  
  --cert=tls.crt \  
  --key=tls.key
```

Secret Types (cont.)

Docker Registry Secret

```
kubectl create secret docker-registry regcred \  
  --docker-server=registry.example.com \  
  --docker-username=user \  
  --docker-password=pass
```

Full Example: ConfigMap + Secret

A realistic app needs both – non-sensitive settings in a ConfigMap, credentials in a Secret.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-config
  namespace: dev
data:
  APP_ENV: production
  LOG_LEVEL: info
  app.properties: |
    server.port=8080
    cache.ttl=300
```

```
apiVersion: v1
kind: Secret
metadata:
  name: webapp-secrets
  namespace: dev
type: Opaque
stringData:
  DB_USER: webapp
  DB_PASS: Tr@iningOnly123
  API_KEY: sk-demo-1234567890
```

Full Example: Consuming Both

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  namespace: dev
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
```

Full Example: Consuming Both (cont.)

```
spec:
  containers:
  - name: webapp
    image: nginx:1.27.2
    envFrom:
    - configMapRef:
        name: webapp-config      # APP_ENV, LOG_LEVEL as env vars
    - secretRef:
        name: webapp-secrets     # DB_USER, DB_PASS, API_KEY as env vars
```

Full Example: Consuming Both (cont. 2)

```
volumeMounts:
- name: config-volume
  mountPath: /etc/config    # app.properties as a file
- name: secret-volume
  mountPath: /etc/secrets
  readOnly: true
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 250m
    memory: 256Mi
volumes:
- name: config-volume
  configMap:
    name: webapp-config
- name: secret-volume
  secret:
    secretName: webapp-secrets
```

Verifying the Full Example

```
# confirm env vars landed in the pod
kubectl exec deploy/webapp -n demo -- env | grep -E 'APP_ENV|DB_USER'

# confirm mounted files from both ConfigMap and Secret
kubectl exec deploy/webapp -n demo -- ls /etc/config /etc/secrets

# decode a secret value manually (base64, not encrypted)
kubectl get secret webapp-secrets -n demo -o jsonpath='{.data.DB_PASS}' | base64 -d
```


Health Checks

Liveness, Readiness, and Startup Probes

Why Health Checks Matter

Without health checks, Kubernetes only knows if a container **process** is running – not if the application inside is **healthy**.

The Problem

- A web server process is running but returning 500 errors
- An app is stuck in a deadlock – process alive, app dead
- A container takes 60 seconds to start, but gets traffic immediately

The Solution: Probes

Probe	Question It Answers	What Happens on Failure
Liveness	Is the app still alive?	Container is restarted
Readiness	Is the app ready to serve traffic?	Pod is removed from Service endpoints
Startup	Has the app finished starting?	Other probes are disabled until it passes

Liveness Probes

If the liveness probe fails, the kubelet **restarts the container**.

HTTP Liveness Probe

```
spec:
  containers:
  - name: app
    image: k8s.gcr.io/echoserver:1.10
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 10    # wait 10s before first check
      periodSeconds: 5          # check every 5s
      failureThreshold: 3       # restart after 3 consecutive failures
```

TCP Liveness Probe

```
image: nginx:1.28
livenessProbe:
  tcpSocket:
    port: 80                    # just checks if the port is open
  periodSeconds: 10
```

Liveness Probes (cont.)

Command Liveness Probe

```
livenessProbe:
  exec:
    command:
      - cat
      - /tmp/healthy          # exit code 0 = healthy
  periodSeconds: 5
```

Readiness Probes

If the readiness probe fails, the pod is **removed from Service endpoints** – it stops receiving traffic but is NOT restarted.

```
spec:
  containers:
  - name: app
    image: myapp:latest
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
      failureThreshold: 3
```

Liveness vs Readiness

	Liveness	Readiness
On failure	Restart container	Remove from Service
Use case	App is dead/stuck	App is temporarily busy
Example	Deadlock detection	Warming cache, loading data

Best practice: Use both. Readiness to protect traffic, liveness to recover from crashes. Use different endpoints if possible (`/healthz` vs `/ready`).

Startup Probes

For applications with **long startup times**, a startup probe prevents the liveness probe from killing the container before it's ready.

```
spec:
  containers:
  - name: app
    image: myapp:latest
    startupProbe:
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30      # 30 attempts
      periodSeconds: 10       # every 10s = up to 300s (5 min) to start
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      periodSeconds: 5
```

How It Works

1. On container start, **only** the startup probe runs
2. Liveness and readiness probes are **disabled** until startup succeeds
3. Once the startup probe passes, it never runs again – liveness/readiness take over

Use startup probes for legacy apps or apps that need to load large datasets on boot.

Common Troubleshooting Commands

Essential kubectl commands for debugging

Debugging Pods

Check Pod Status

```
kubectl get pods           # quick status overview
kubectl get pods -o wide    # show node placement and IP
kubectl describe pod <pod-name> # detailed events and conditions
```

Common Pod States

Status	Meaning
Pending	Pod accepted but not yet scheduled (resource constraints, node issues)
ContainerCreating	Image being pulled or volumes being mounted
Running	All containers started
CrashLoopBackOff	Container keeps crashing and restarting
ImagePullBackOff	Cannot pull the container image
ErrImagePull	Failed to pull image (wrong name, no auth, network)
Completed	Container exited successfully (exit code 0)

Reading Logs

```
# view logs for a pod
kubectl logs <pod-name>

# follow logs in real-time (like tail -f)
kubectl logs <pod-name> -f

# view logs from a previous container instance (after a crash)
kubectl logs <pod-name> --previous

# view logs from a specific container in a multi-container pod
kubectl logs <pod-name> -c <container-name>

# view last 50 lines
kubectl logs <pod-name> --tail=50

# view logs from the last 5 minutes
kubectl logs <pod-name> --since=5m
```

Exec into a Container

```
# open an interactive shell
kubectl exec -it <pod-name> -- /bin/bash

# run a single command
kubectl exec <pod-name> -- env

# exec into a specific container in a multi-container pod
kubectl exec -it <pod-name> -c <container-name> -- /bin/sh
```

Useful Debug Commands Inside the Container

```
# check environment variables
env | grep DB

# check DNS resolution
nslookup kubernetes.default

# test connectivity to another service
curl http://my-service:8080/health

# check if a file was mounted
ls -la /etc/config/
cat /etc/config/app.properties
```

Inspecting Events

Events tell you **what happened** in the cluster – scheduling decisions, image pulls, probe failures, etc.

```
# all events in the current namespace (sorted by time)
kubectl get events --sort-by='.lastTimestamp'

# events for a specific pod (shown at the bottom of describe)
kubectl describe pod <pod-name>

# all events across the cluster
kubectl get events -A

# watch events in real-time
kubectl get events -w
```

Common Events to Watch For

- `FailedScheduling` – no node has enough resources
- `BackOff` – container keeps crashing
- `FailedMount` – volume or secret not found
- `Unhealthy` – liveness/readiness probe failed
- `Pulling` / `Pulled` – image pull status

Quick Reference: Troubleshooting Workflow

When a pod is not working, follow this order:

- | | |
|--|--|
| 1. <code>kubectl get pods</code> | → What state is the pod in? |
| 2. <code>kubectl describe pod <name></code> | → What do the events say? |
| 3. <code>kubectl logs <name></code> | → What does the app say? |
| 4. <code>kubectl logs <name> --previous</code> | → Did it crash? What were the last logs? |
| 5. <code>kubectl exec -it <name> -- sh</code> | → Can I poke around inside? |
| 6. <code>kubectl get events --sort-by=time</code> | → What's happening cluster-wide? |

Is it a scheduling problem?

```
kubectl describe pod <name> | grep -A5 "Events"
```

Look for: FailedScheduling, Insufficient cpu, Insufficient memory

Is it an image problem?

```
kubectl describe pod <name> | grep -A3 "State"
```

Look for: ImagePullBackOff, ErrImagePull

Is it a config problem?

```
kubectl describe pod <name> | grep -A3 "Environment"
```

Check mounted ConfigMaps/Secrets

Hands-On Labs

Deploying, updating, configuring, and troubleshooting

Lab 1: Deploy a Multi-Replica Application

Create a Deployment

```
kubectl create deployment web --image=httpd --replicas=3 \
--dry-run=client -o yaml > web-deployment.yaml
```

Edit `web-deployment.yaml` and add resource requests:

```
resources:
  requests:
    cpu: "50m"
    memory: "64Mi"
  limits:
    memory: "128Mi"
```

```
kubectl apply -f web-deployment.yaml
```

Verify

```
kubectl get deployment web
kubectl get rs
kubectl get pods -l app=web -o wide
```

Lab 2: Rolling Updates and Rollbacks

Perform a Rolling Update

```
# update the image to a newer version
kubectl set image deployment/web httpd=httpd:2.4.68

# watch the rollout
kubectl rollout status deployment/web

# check ReplicaSets – you should see two (old and new)
kubectl get rs
```

Trigger a Bad Update

```
# update to a non-existent image
kubectl set image deployment/web httpd=httpd:9.99.99

# watch it fail
kubectl rollout status deployment/web
kubectl get pods
```

Lab 2: Rolling Updates and Rollbacks (cont.)

Roll Back

```
# revert to the last working version
kubectl rollout undo deployment/web

# verify recovery
kubectl rollout status deployment/web
kubectl get pods
```

Lab 3: Expose the Application

Create a Service

```
kubectl expose deployment web --port=80 --type=ClusterIP \
  --dry-run=client -o yaml > web-service.yaml
```

```
kubectl apply -f web-service.yaml
```

Test the Service

```
# get the ClusterIP
kubectl get svc web

# test from within the cluster
kubectl run curl --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://web.default.svc.cluster.local
```

Expose via NodePort

```
kubectl expose deployment web --port=80 --type=NodePort --name=web-nodeport
kubectl get svc web-nodeport
```

The NodePort is accessible on <any-node-ip>:<nodeport> .

Lab 4: ConfigMaps and Secrets

Create a ConfigMap

```
kubectl create configmap web-config \  
  --from-literal=WELCOME_MSG="Hello from Day 2!" \  
  --from-literal=APP_COLOR=blue
```

Create a Secret

```
kubectl create secret generic web-secret \  
  --from-literal=API_KEY=my-secret-api-key-12345
```

Verify

```
kubectl get configmap web-config -o yaml  
kubectl get secret web-secret -o yaml  
  
# decode a secret value  
kubectl get secret web-secret -o jsonpath='{.data.API_KEY}' | base64 -d
```

Lab 4: ConfigMaps and Secrets (cont.)

Consume in a Pod

Create `configmap-pod.yaml` :

```
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
spec:
  containers:
    - name: demo
      image: busybox
      command: ["sh", "-c", "echo $WELCOME_MSG && echo API_KEY=$API_KEY && sleep 3600"]
      envFrom:
        - configMapRef:
            name: web-config
      env:
        - name: API_KEY
          valueFrom:
            secretKeyRef:
              name: web-secret
              key: API_KEY
```

Lab 4: ConfigMaps and Secrets (cont.)

```
kubectl apply -f configmap-pod.yaml  
kubectl logs config-demo
```

Expected output: Hello from Day 2! and API_KEY=my-secret-api-key-12345 .

Lab 5: Health Checks

Create a Pod with Probes

Create `probe-demo.yaml` :

Lab 5: Health Checks

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-demo
spec:
  containers:
  - name: app
    image: busybox
    command:
    - sh
    - -c
    - |
      touch /tmp/healthy
      echo "App started"
      sleep 30
      rm /tmp/healthy
      echo "App became unhealthy"
      sleep 600
  livenessProbe:
    exec:
      command: ["cat", "/tmp/healthy"]
    initialDelaySeconds: 5
    periodSeconds: 5
```

Lab 5: Health Checks (cont.)

```
readinessProbe:  
  exec:  
    command: ["cat", "/tmp/healthy"]  
  initialDelaySeconds: 5  
  periodSeconds: 5
```

Lab 5: Health Checks (cont.)

Deploy and Observe

```
kubectl apply -f probe-demo.yaml
```

```
# watch the pod – after ~30s the file is deleted  
kubectl get pods -w
```

What You'll See

1. Pod starts, probes pass – pod is `Running` and `1/1 READY`
2. After 30 seconds, `/tmp/healthy` is removed
3. Readiness probe fails – pod becomes `0/1 READY` (removed from endpoints)
4. Liveness probe fails 3 times – container is **restarted**
5. After restart, `/tmp/healthy` is recreated – pod becomes `1/1 READY` again

```
# check the events  
kubectl describe pod probe-demo | grep -A10 Events
```

Look for Unhealthy events with Liveness probe failed and Readiness probe failed messages.

Lab 6: Troubleshoot a Broken Pod

Deploy a Broken Pod

```
kubectl run broken --image=nginx:nonexistent --port=80
```

Investigate

```
# 1. What state is the pod in?
```

```
kubectl get pods
```

```
# 2. What do the events say?
```

```
kubectl describe pod broken
```

```
# 3. What are the logs?
```

```
kubectl logs broken
```

Fix It

```
kubectl set image pod/broken broken=nginx:1.27
```

Note: You can't edit most fields on a bare pod. In practice, use a Deployment – then you'd just update the Deployment spec and it handles the replacement.

Lab 7: Namespaces in Practice

Create Namespaces

```
kubectl create namespace staging  
kubectl create namespace production
```

Deploy to Each Namespace

```
kubectl create deployment web --image=nginx:1.27 --replicas=2 -n staging  
kubectl create deployment web --image=nginx:1.27 --replicas=3 -n production
```

Compare

```
# same name, different namespaces, different replica counts  
kubectl get deployment web -n staging  
kubectl get deployment web -n production  
  
# see everything across namespaces  
kubectl get deployments -A
```

Clean Up

```
# deleting a namespace deletes everything inside it  
kubectl delete namespace staging  
kubectl delete namespace production
```

Day 2 Recap

What We Covered Today

-  Namespaces – organizing and isolating resources
-  Pods, ReplicaSets, Deployments – the workload resource hierarchy
-  Labels and Selectors – how Kubernetes connects resources
-  Rolling Updates and Rollbacks – zero-downtime deployments
-  Scaling – manual scaling and Horizontal Pod Autoscaler
-  Resource Requests and Limits – CPU and memory management
-  Declarative vs Imperative – when to use each approach
-  ConfigMaps and Secrets – externalizing configuration
-  Health Checks – liveness, readiness, and startup probes
-  Troubleshooting – systematic debugging with kubectl

Coming Up Tomorrow

Day 3: Networking, Scheduling, and Storage

- Kubernetes Networking – Cluster, Pod, and Service networking
- DNS and CoreDNS
- Ingress and Gateway API
- Scheduling – Taints, Tolerations, and Node Selectors
- DaemonSets, Jobs, and CronJobs
- Persistent Volumes and StorageClasses

Labs

- Configuring Services and Ingress
- Workload scheduling and node targeting
- Persistent storage configuration with Linode CSI
- Creating LoadBalancer services with Linode CCM

Questions?